

Uncertainty-Aware Persistent 3D World Models: Student-Scale Framework Replication and Extension

CS 599 G1 (Spring 2026) — Deep Visual Generative Models — Final Paper

Jitarth
Boston University
jitarth@bu.edu
BU ID: U05789425

Abstract

Long-horizon *visual* prediction—a core theme in deep generative modeling for video—requires a model to remember previously observed 3D scene content across time. The course reference manuscript *Learning 3D Persistent Embodied World Models* proposes persistent 3D memory together with a diffusion-based generative world model to forecast RGB-D observations from an embodied agent’s experience [1]. Faithfully reproducing that full diffusion-based architecture and training regime was outside the compute budget of this semester-long project.

We instead replicate the *core persistent-memory framework* at student scale: from context RGB-D frames and camera parameters we lift pixels into 3D, accumulate a dense voxel memory, render the memory into each target viewpoint, condition future-frame prediction on those renders, roll out multi-step predictions, and update memory over the rollout. To keep experiments tractable while isolating memory and uncertainty effects, we use a *ConvGRU* spatiotemporal backbone rather than a full diffusion/DiT generator.

Our extension estimates per-pixel uncertainty (log-variance), maps it to confidence, and applies confidence-gated memory writes so that unreliable generated pixels are less likely to corrupt the persistent buffer. On a 200-clip MOVi-A subset (320 validation windows; uniform evaluation harness), the memory-conditioned ConvGRU improves over a no-memory ConvGRU on validation L1 ($0.0214 \rightarrow 0.0188$; -12.1%), dynamic-region L1 ($0.1023 \rightarrow 0.0890$; -13.0%), and PSNR ($25.57 \rightarrow 26.68$ dB). Adding uncertainty-gated writes further reduces memory-covered L1 ($0.0577 \rightarrow 0.0443$; -23.2%), with uncertainty positively correlated to per-pixel error ($r=0.150$). These results support uncertainty-aware memory updates as a practical way to improve the *reliability* of persistent geometric memory in student-scale world models.

Course alignment. This project studies future visual prediction with explicit 3D memory, connects to diffusion-based generative world modeling through the reference manuscript, and analyses behaviour quantitatively and qualitatively—while being honest that the primary experiments use a compute-feasible recurrent backbone.

1 Introduction

Predicting future RGB views of a 3D scene is a central problem in *generative* video modeling: the model must hallucinate plausible temporal evolution while respecting geometry, motion, and visibility. When the camera or objects move, purely frame-local predictors forget distant scene content; recurrent predictors helped by context still struggle as errors compound over time. The reference manuscript on persistent 3D embodied world models [1] argues for an *explicit, geometrically grounded* memory into which observations are written and from which novel views can be queried, paired with a diffusion-style generative predictor.

Scope and objective. Our objective is **not** exact replication of every component of that architecture at original scale, but **mechanism replication**: we isolate the *memory write, read/render, memory-conditioned prediction, and memory update* loop that defines the framework, and we test a focused extension on top.

Although the original paper uses a diffusion-based generator, reproducing that component was outside the compute budget of this course project. We therefore use ConvGRU as a compute-feasible backbone to isolate and evaluate the persistent-memory mechanism and our uncertainty-gated memory update extension. This substitution changes the generator, but preserves the memory-write, memory-read, and memory-update loop that defines the original framework. The substitution of ConvGRU for the original diffusion backbone is a compute-driven simplification, *not* a claim that ConvGRU is a better generative model.

Research question. We ask:

Does persistent 3D memory improve future-frame prediction at student scale, and can uncertainty-gated memory updates improve the reliability of that memory?

Contributions.

1. **Framework replication.** A student-scale implementation of persistent 3D voxel memory with target-view rendering, memory-conditioned rollout, and updates—aligned with the reference manuscript’s loop but without claiming full architecture match.
2. **Controlled ConvGRU study.** Three matched variants: no-memory, memory-conditioned, and memory-conditioned with uncertainty-gated writes, under identical data splits and comparable training budgets.
3. **Uncertainty-gated memory updates.** Heteroscedastic uncertainty with confidence thresholding to limit unreliable generated content from being written into memory.
4. **Evaluation harness.** Unified validation metrics plus diagnostic slices (high motion, occlusion recovery, high memory coverage, depth-edge-heavy).

2 Related Work

2.1 Persistent 3D world models

The course reference manuscript [1] frames embodied world modeling as maintaining a persistent 3D scene representation: RGB-D observations are fused into memory using known intrinsics and extrinsics; the model predicts future RGB-D conditioned on pose and on memory readouts; memory evolves as the agent acts and predicts. Our work preserves this *interaction pattern* at reduced scale.

2.2 Video prediction and recurrent visual models

Convolutional recurrent cells (ConvLSTM [3], ConvGRU [4]) are standard for deterministic or lightly stochastic video prediction. ConvGRU offers a lightweight, single-pass spatiotemporal backbone suitable for controlled experiments when diffusion-scale training is infeasible.

2.3 Diffusion video models

Conditional video diffusion is the dominant modern family for high-fidelity generative video [5] and matches the *spirit* of the large generative backbone in the reference manuscript. We did not treat full diffusion/DiT reproduction as the main project deliverable because of compute and engineering cost; we position a diffusion-style branch as **future work**, not as a failed baseline.

2.4 Uncertainty-aware prediction

Heteroscedastic uncertainty and per-pixel variance modeling [6] let a network signal where predictions are unreliable. We reuse this machinery both as an auxiliary loss and as a confidence map for selectively writing predicted views into memory.

2.5 MOVi / Kubric

MOVi-A (Kubric [2]) provides RGB-D video, camera intrinsics, and poses in synthetic scenes with clean geometry—useful for testing unprojection, voxel fusion, and render/read correctness before moving to noisy real-world embodied data.

3 Replication Scope: Framework, Not Full Architecture

Table 1 summarises what we replicate versus what we simplify. The intent is to prevent misreading this report as claiming an exact, full-scale reproduction of the reference manuscript.

Original paper component	Our implementation	Status
Persistent 3D memory	Dense voxel memory from RGB-D and poses	Replicated at student scale
RGB-D observations	MOVi-A RGB-D clips	Replicated
Camera pose / action conditioning	Camera and relative target-pose conditioning	Partially replicated
2D-to-3D lifting	Unprojection using intrinsics, pose, and depth	Replicated
Memory read / render	Target-view memory render and coverage mask	Replicated
Memory-conditioned prediction	ConvGRU conditioned on rendered memory	Simplified but replicated
Diffusion / DiT generator	Not used as primary model (compute budget)	Not fully replicated
DINO / semantic feature memory	Not used (RGB-focused memory)	Simplified
Large embodied training	MOVi-A subset (200 clips)	Simplified
Uncertainty-gated writes	Confidence-thresholded updates for predicted frames	Added (our extension)

Table 1: What we replicate versus what we simplify. We target *mechanism replication* of the persistent-memory loop, not bit-for-bit match to the reference implementation.

4 Task Setup and Notation

Let a clip provide RGB frames $\{I_t\}_{t=1}^T$, depth maps $\{D_t\}_{t=1}^T$, camera poses $\{P_t\}_{t=1}^T$ (each a **camera-to-world** transform, as in Kubric / our loader), and intrinsics K . We split each clip into windows of length $T_c + T_p$: the first T_c frames are *context* and the next T_p frames are *targets* to predict.

The model receives context RGB $\{I_t\}_{t=1}^{T_c}$, context poses $\{P_t\}_{t=1}^{T_c}$, and target poses $\{P_t\}_{t=T_c+1}^{T_c+T_p}$, and outputs predicted RGB $\{\hat{I}_t\}_{t=T_c+1}^{T_c+T_p}$. Memory variants additionally consume a rendered memory tensor per target frame (Section 5.4).

MOVi clips are exported at 128×128 ; we train and evaluate at 64×64 . All final runs use $T_c = T_p = 4$.

5 Method

5.1 Overall framework

At a high level, each rollout executes: *build memory from context* $\rightarrow$ *render memory at target poses* $\rightarrow$ *predict future RGB (and auxiliary depth / uncertainty as enabled)* $\rightarrow$ *update memory* along the predicted trajectory. Figure 1 shows the ConvGRU-based instantiation; Table 2 clarifies how this differs from the diffusion-based generator in the reference manuscript.

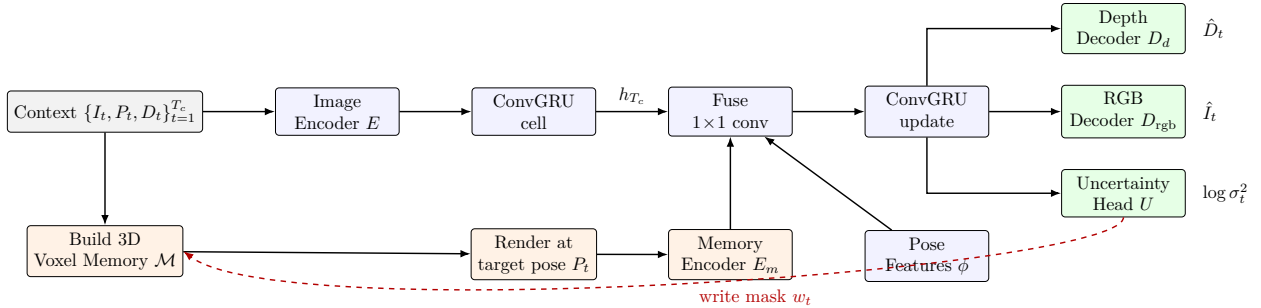


Figure 1: ConvGRU-based instantiation of the persistent-memory framework used for all main results. Solid arrows: forward pass; dashed red path: uncertainty-to-memory write gating (uncertainty variant only). This diagram reflects our student-scale backbone, not the diffusion/DiT generator in the reference manuscript.

Aspect	Reference diffusion-style backbone	Our ConvGRU backbone
Generator type	Stochastic generative model (diffusion / iterative denoising)	Deterministic recurrent predictor (one forward pass)
Typical compute / scale	High (long training, large models)	Lower (student-scale budgets)
Faithfulness to paper generator	High	Simplified (not a drop-in replacement)
Role in this project	Future-work direction	primary controlled experiments

Table 2: The ConvGRU backbone is a *substitute* for tractable experiments, not a claim of superiority over diffusion.

5.2 Data pipeline

We export MOVi-A clips into compressed `.npz` shards containing RGB, depth, poses, intrinsics, and metadata. A manifest lists shard paths for training scripts. We sample overlapping context/target windows with capped counts per split and a fixed `val_ratio=0.2`, so all models see identical splits.

5.3 3D memory construction

Let $P \in \text{SE}(3)$ denote the **camera-to-world** pose, stored in the codebase as a 4×4 matrix `camera_to_world` (same convention as Kubric exports: translation plus rotation from camera axes to world axes). Let $\tilde{\mathbf{u}} = [u, v, 1]^\top$. A camera-frame 3D point obtained by back-projecting (u, v) with intrinsics K and depth $D_t(u, v)$ is $\mathbf{x}_c = D_t(u, v) K^{-1} \tilde{\mathbf{u}}$ in the usual pinhole *z-depth* model. MOVi/Kubric depth is *radial* along a viewing ray; the implementation equivalently forms $\mathbf{x}_c$ by scaling a unit ray from K (`depth_to_camera_points`) before applying P . The corresponding world point is

$$\tilde{\mathbf{X}}_w = P \tilde{\mathbf{X}}_c, \quad \tilde{\mathbf{X}}_c = \begin{bmatrix} \mathbf{x}_c \\ 1 \end{bmatrix}. \quad (1)$$

For the textbook identity $\mathbf{x}_c = D_t(u, v) K^{-1} \tilde{\mathbf{u}}$, this is equivalently

$$\tilde{\mathbf{X}}_w = P \begin{bmatrix} D_t(u, v) K^{-1} [u, v, 1]^\top \\ 1 \end{bmatrix}. \quad (2)$$

A **world-to-camera** pose would appear as P^{-1} on world points; we do *not* define P that way here, so there is no P^{-1} in the unprojection—only P mapping camera coordinates forward into the world frame.

We voxelise these world points on a fixed grid (resolution $48 \times 40 \times 48$) by splatting RGB from context frames into $\mathcal{M}$.

5.4 Memory rendering and reading

For each target **camera-to-world** pose P_t , we render $\mathcal{M}$ into a target-aligned memory tensor R_t , a coverage mask $m_t^{\mathcal{M}}$ (where the render is supported), and a rendered depth. A memory encoder E_m maps R_t to features compatible with the image encoder for fusion.

5.5 ConvGRU prediction backbone

A ConvGRU [4] maintains a spatial hidden state and is updated once per predicted timestep. We encode each context frame, unroll over context to obtain h_{T_c} , then for each target time fuse image features, memory features, and pose embeddings. Warm-starting (memory from no-memory; uncertainty from memory) keeps optimisation stable.

5.6 Baselines

No-memory ConvGRU. Predicts future RGB from encoded context and pose only. Training uses RGB L1 plus dynamic-region L1.

Memory-conditioned ConvGRU. The model adds $E_m(R_t)$ to fusion and depth decoding. Training adds memory-covered L1 so optimisation prefers predictions that agree with the memory render where it is defined.

5.7 Extension: uncertainty-gated memory writes

Let h_t denote features at target time t . The RGB head implements a prediction map; we write abstractly

$$\hat{I}_t = f_\theta(h_t, \text{pose}, \text{memory}). \quad (3)$$

The uncertainty head predicts per-pixel log-variance / scores

$$s_t(u, v) = (U(h_t))(u, v), \quad (4)$$

with $\log \sigma_t^2 = \text{clip}(U(h_t))$ for numerical stability. Training adds Gaussian negative log-likelihood (Section 5.8) with $\bar{\delta}_t^2 = \frac{1}{3} \sum_c (\hat{I}_t^{(c)} - I_t^{(c)})^2$.

Confidence and a binary write mask are

$$c_t = \frac{e^{-\log \sigma_t^2}}{1 + e^{-\log \sigma_t^2}}, \quad \tilde{c}_t = c_t^\gamma, \quad w_t = \mathbb{1}[\tilde{c}_t \geq \tau], \quad (5)$$

with $\gamma = 4.0$ and $\tau = 0.99$. **Observed context** frames are written into memory with full trust; **generated** RGB from rollout is written only where $w_t=1$, so unreliable predictions are less likely to corrupt $\mathcal{M}$. Abstractly, letting v index voxels projected to pixel (u, v) ,

$$\mathcal{M}_{t+1}(v) = \begin{cases} \text{update}(\mathcal{M}_t(v), \text{predicted colour}) & \text{if } w_t(u, v) = 1 \\ \mathcal{M}_t(v) & \text{otherwise.} \end{cases} \quad (6)$$

5.8 Encoder details, recurrence, and loss functions

Encoder and ConvGRU. Each frame is encoded by $E : \mathbb{R}^{3 \times H \times W} \rightarrow \mathbb{R}^{C \times h \times w}$ with $C=128$. ConvGRU gates follow [4]; we unroll over context to h_{T_c} .

Relative pose features. For anchor pose P_a (last context pose) and target P_t , $\phi(P_a, P_t) = \text{vec}((P_a^{-1}P_t)_{[3,4]}) \in \mathbb{R}^{12}$, passed through an MLP and broadcast spatially.

Target-time recurrence (memory variants).

$$g_t = \text{Conv}_{1 \times 1}([E(\hat{I}_{t-1}), E_m(R_t), \phi(P_a, P_t)]), \quad h_t = \text{ConvGRU}(g_t, h_{t-1}), \quad (7)$$

$$\hat{I}_t = \text{clip}_{[0,1]}(\hat{I}_{t-1} + \alpha \tanh(D_{\text{rgb}}(h_t))), \quad \hat{D}_t = \text{ReLU}(D_d(h_t)), \quad (8)$$

with $\alpha = 0.25$, initialising $\hat{I}_{T_c} = I_{T_c}$. The no-memory baseline omits $E_m(R_t)$ and uses a simpler fusion of $E(\hat{I}_{t-1})$ with pose.

Masks and composite loss. Dynamic mask threshold $\theta = 0.03$. Losses:

$$\mathcal{L}_{\text{rgb}} = \frac{1}{T_p H W} \sum_{t,c,u,v} |I_t - \hat{I}_t|, \quad (9)$$

$$\mathcal{L}_{\text{dyn}} = \text{MaskedL1}(\hat{I}, I; m^{\text{dyn}}), \quad (10)$$

$$\mathcal{L}_{\text{depth}} = \text{MaskedL1}(\hat{D}, D; \mathbb{1}[D > 0]), \quad (11)$$

$$\mathcal{L}_{\mathcal{M}} = \text{MaskedL1}(\hat{I}, I; m^{\mathcal{M}}), \quad (12)$$

$$\mathcal{L}_{\text{NLL}} = \frac{1}{T_p H W} \sum_{t,u,v} \left[\frac{1}{2} e^{-\log \sigma_t^2} \bar{\delta}_t^2 + \frac{1}{2} \log \sigma_t^2 \right], \quad (13)$$

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{rgb}} + \lambda_{\text{dyn}} \mathcal{L}_{\text{dyn}} + \lambda_{\text{depth}} \mathcal{L}_{\text{depth}} + \lambda_{\mathcal{M}} \mathcal{L}_{\mathcal{M}} + \lambda_{\text{NLL}} \mathcal{L}_{\text{NLL}}. \quad (14)$$

Weights: $\lambda_{\text{dyn}} = 3$ always; $\lambda_{\text{depth}} = 0.1$, $\lambda_{\mathcal{M}} = 1$ for memory variants; $\lambda_{\text{NLL}} = 0.25$ for the uncertainty variant. No-memory keeps only $\mathcal{L}_{\text{rgb}}$ and $\mathcal{L}_{\text{dyn}}$.

6 Experimental Setup

Dataset and motivation. MOVi-A (Kubric [2]) offers RGB-D video with intrinsics and poses in controlled synthetic scenes, which supports geometric debugging of unprojection and voxel memory. We use 200 clips at source resolution 128×128 , split into 160 train / 40 validation clips with a fixed seed. Up to 16 train and 8 validation windows per clip yield 2,560 train windows and 320 validation windows. Each window uses $T_c = T_p = 4$; models consume RGB at 64×64 .

Models compared (main results). Three ConvGRU variants: (1) no-memory; (2) memory-conditioned; (3) memory with uncertainty-gated writes. We do **not** include full diffusion/DiT reproduction in the main quantitative claims.

Architecture. Hidden channels 128; voxel grid (48, 40, 48), stride 1, splat radius 1 for memory variants.

Training. No-memory and memory: 50 epochs, batch size 16, Adam lr 10^{-3} . Uncertainty: up to 40 epochs, warm-started from the memory checkpoint. Checkpoints are selected by best validation L1.

Loss hyperparameters. Dynamic weight 3.0; depth weight 0.1; memory-covered weight 1.0; memory render loss weight 0.0; uncertainty NLL weight 0.25; $\gamma = 4.0$, $\tau = 0.99$.

Hardware. Training and evaluation used single-GPU workstations and cluster GPUs (e.g., L40S-class). Memory conditioning dominates runtime versus the no-memory baseline.

7 Evaluation Protocol

We load each model’s best checkpoint on the same 320 validation windows.

Metrics (why they matter).

- **Val L1** ($\downarrow$): overall RGB reconstruction error—global prediction quality.
- **Dynamic L1** ($\downarrow$): error on moving/changing pixels—stress-tests temporal change.
- **Memory-covered L1** ($\downarrow$): error where oracle memory render is non-empty—whether memory-supported regions improve when memory is in play.
- **PSNR** ($\uparrow$): logarithmic quality summary correlated with perceptual sharpness (within limits at 64×64).
- **Uncertainty-error correlation** (uncertainty model): tests whether s_t tracks actual error, supporting gating.
- **Write coverage**: mean fraction of pixels passing the gate under τ, γ —how selective writes are at runtime.

Diagnostic slices. We aggregate windows with high motion, occlusion-recovery semantics, high memory coverage, and depth-edge density (top fractions within validation). On our MOVi-A subset the camera is effectively static, so a **high camera motion** slice is empty and omitted—this limits conclusions about viewpoint-heavy generalisation.

8 Results

8.1 Does persistent memory help?

Table 3 compares the no-memory ConvGRU to the memory-conditioned ConvGRU. Memory conditioning uniformly improves validation L1, dynamic L1, memory-covered L1, and PSNR—consistent with the hypothesis that explicit 3D memory helps future RGB prediction even when the generator is a compact recurrent model rather than a diffusion stack.

Model	Val L1 ↓	Dyn L1 ↓	Mem-cov L1 ↓	PSNR (dB) ↑	Unc-corr	Write-cov
ConvGRU no-memory	0.0214	0.1023	0.0653	25.57	—	—
ConvGRU memory	0.0188	0.0890	0.0577	26.68	—	0.000
ConvGRU memory + uncertainty	0.0185	0.0899	0.0443	26.71	0.150	0.973

Table 3: Unified validation metrics (320 windows). Metrics read from the unified evaluation JSON file under `outputs/eval_final/` (checkpoints trained under `outputs/local_movia200_64_convgru*_v1`).

8.2 Does uncertainty-gated writing help?

Comparing the memory model to memory+uncertainty, the largest targeted gain is on **memory-covered L1** ($0.0577 \rightarrow 0.0443$; -23.2%), the metric most sensitive to whether writes and reads maintain faithful geometry in memory-supported regions. Overall L1 and PSNR improve slightly; dynamic L1 moves by 0.0009 (small regression, likely within optimisation noise at 64×64). Uncertainty correlates positively with error ($r=0.150$), indicating signal usable for gating despite weak absolute correlation. Write coverage 0.973 shows the threshold leaves most pixels writable while still allowing sparse suppressions where confidence drops—the mask pattern in Figure 6 matters more than the scalar mean alone.

8.3 Slice-level evaluation

Table 4 reports L1 on diagnostic slices. The ranking $\text{uncertainty} \leq \text{memory} < \text{no-memory}$ holds on each listed slice, with the uncertainty variant best on all non-trivial slices.

Slice (count)	No-memory	Memory	Memory + uncertainty
all (320)	0.0214	0.0188	0.0185
high_motion (80)	0.0295	0.0263	0.0260
occlusion_recovery (37)	0.0232	0.0205	0.0203
high_memory_coverage (80)	0.0287	0.0259	0.0256
depth_edge_heavy (80)	0.0276	0.0246	0.0243

Table 4: Validation L1 by slice. *high_camera_motion* is omitted (empty) because cameras are effectively static on this subset.

8.4 Qualitative results

Although predictions remain blurry at 64×64 , memory-conditioned and uncertainty-gated variants align better with the quantitative trends: lower error in dynamic and memory-covered regions. Figures 2–5 summarise representative windows; they are intended as interpretability aids, not photorealistic generation claims.

8.5 Uncertainty maps and write masks

Figure 6 isolates per-pixel confidence and the binary write mask for one validation window. Brighter confidence means the model self-reports higher certainty (lower predicted variance). The write mask is white where $\tilde{c}_t \geq \tau$ and black where updates to $\mathcal{M}$ from *generated* pixels are suppressed. Context observations are still written with full trust; the mask primarily regulates contamination from uncertain hallucinated content during rollout.

8.6 Diffusion backbone not fully reproduced

The reference manuscript centres a diffusion-style generative predictor. Training a comparable conditional video diffusion/DiT model to convergence would require substantially more GPU time and tuning than budgeted here. Repository scripts document a lightweight diffusion branch for *exploratory* runs; we do *not* report diffusion metrics as main results, and we do not frame diffusion as a “failed” baseline—it remains principled future work with adequate compute.

9 Discussion

Framework replication. We reproduced the persistent memory *loop*—write, render/read, predict, update—at a scale compatible with a graduate course project, without asserting a full match to every module in the reference manuscript.

Memory helps. Memory-conditioned ConvGRU improves over no-memory on standard and memory-centric metrics, supporting the claim that geometric memory matters for student-scale video forecasting even when the generator is not diffusion-sized.

Uncertainty and memory reliability. Uncertainty-gated writing primarily improves memory-covered L1, which is exactly where memory should pay off. In persistent world models, prediction errors are not isolated: once written into memory, they can bias future reads. Selective writes mitigate this failure mode.

Calibration and thresholds. γ and τ matter because sigmoid confidence can saturate; the operating point balances coverage and selectivity (Table 3).

Compute-feasible backbone. ConvGRU was selected as the primary backbone because it enabled controlled experiments under available compute, not because it is generically superior to diffusion for video generation.

Diffusion as future work. A stronger diffusion/DiT predictor paired with the *same* memory loop is the natural next step when GPU budget allows.

10 Limitations

- **Not full architecture replication:** no full diffusion/DiT generator at reference scale, no DINO-style semantic feature memory, no large-scale embodied training as in the manuscript’s intended regime.
- **Compute:** diffusion-scale training was out of scope for final claims.
- **Dataset:** MOVi-A is synthetic and controlled; transfer to Habitat, HM3D, or Real-World Robotics is not established here.
- **Resolution:** 64×64 caps detail and qualitative fidelity.
- **Uncertainty:** informative correlation, not guaranteed calibration for arbitrary deployments.
- **Memory representation:** dense voxels do not encode explicit object semantics.
- **Camera motion:** static cameras in our subset mean weak stress tests for viewpoint change.

11 Conclusion and Future Work

We presented a student-scale *framework* replication of persistent 3D world modeling with an uncertainty-gated memory update extension. We do *not* claim to reproduce the full diffusion-based generator from the reference manuscript. Empirically, persistent memory improves a ConvGRU forecaster over a no-memory baseline, and uncertainty-gated writes improve memory-covered prediction—our main proxy for memory reliability.

Future work includes: (i) scaling the generative backbone toward diffusion/DiT while retaining the same memory interface; (ii) richer MOVi splits or MOVi-C/D-style diversity; (iii) evaluation on embodied RGB-D benchmarks with real camera motion; (iv) object-level or semantic uncertainty and feature memory; (v) calibrating confidence maps for principled threshold selection.

12 Reproducibility

- Environment and data export: see `README.md`.
- **Main results pipeline:** `scripts/train_convgru_all.sh`; evaluation via `scripts/eval_all.py` produces `outputs/eval_final/evaluation.json` (numbers in Table 3).
- **Local checkpoint directories** used to build Table 3: `outputs/local_movia200_64_convgru_nomemory_v1`; `outputs/local_movia200_64_convgru_memory_v1`; `outputs/local_movia200_64_convgru_uncertainty_v1`. Table metrics correspond to evaluating these **local** checkpoints (the archived `outputs/eval_final/evaluation.json`); `outputs/scc_movia200_64_convgru_*` paths name separate cluster training runs and are not the source of the quoted numbers unless evaluation is re-run with those directories as the checkpoint roots.
- **Optional cluster workflow:** on shared clusters, the same stages can be orchestrated with SGE scripts `scripts/scc/qsub_train_convgru_all.sh`, `scripts/scc/qsub_train_diffusion_all.sh`, `scripts/scc/qsub_eval_compare.sh`, and `scripts/scc/qsub_make_visuals.sh` when present in a checkout. The ConvGRU script reproduces the main quantitative pipeline; the diffusion script is optional exploratory / future-work tooling.

- **Typical cluster output layout (if used):** `outputs/scc_movia200_64_convgru_nomemory_v1`; `outputs/scc_movia200_64_convgru_memory_v1`; `outputs/scc_movia200_64_convgru_uncertainty_v1`; `outputs/scc_final_eval_movia200_64_v1`; `outputs/scc_final_model_comparison_movia200_64_v1`.
- **Panels:** `scripts/make_visual_comparison_panel.py`; figures under `outputs/report_figures/`.
- **Exploratory diffusion:** `scripts/train_diffusion_all.sh` (not used for main claims in this paper).

References

- [1] *Learning 3D Persistent Embodied World Models*. Course reference manuscript, 2025.
- [2] K. Greff, F. Belletti, L. Beyer, et al. “Kubric: A Scalable Dataset Generator.” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [3] X. Shi, Z. Chen, H. Wang, D.-Y. Yeung, W.-K. Wong, and W.-c. Woo. “Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting.” *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [4] N. Ballas, L. Yao, C. Pal, and A. Courville. “Delving Deeper into Convolutional Networks for Learning Video Representations.” *International Conference on Learning Representations (ICLR)*, 2016.
- [5] J. Ho, T. Salimans, A. Gritsenko, W. Chan, M. Norouzi, and D. Fleet. “Video Diffusion Models.” *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [6] A. Kendall and Y. Gal. “What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?” *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

normal_case: 00006.npz @ frame 0
Median example from all validation windows.

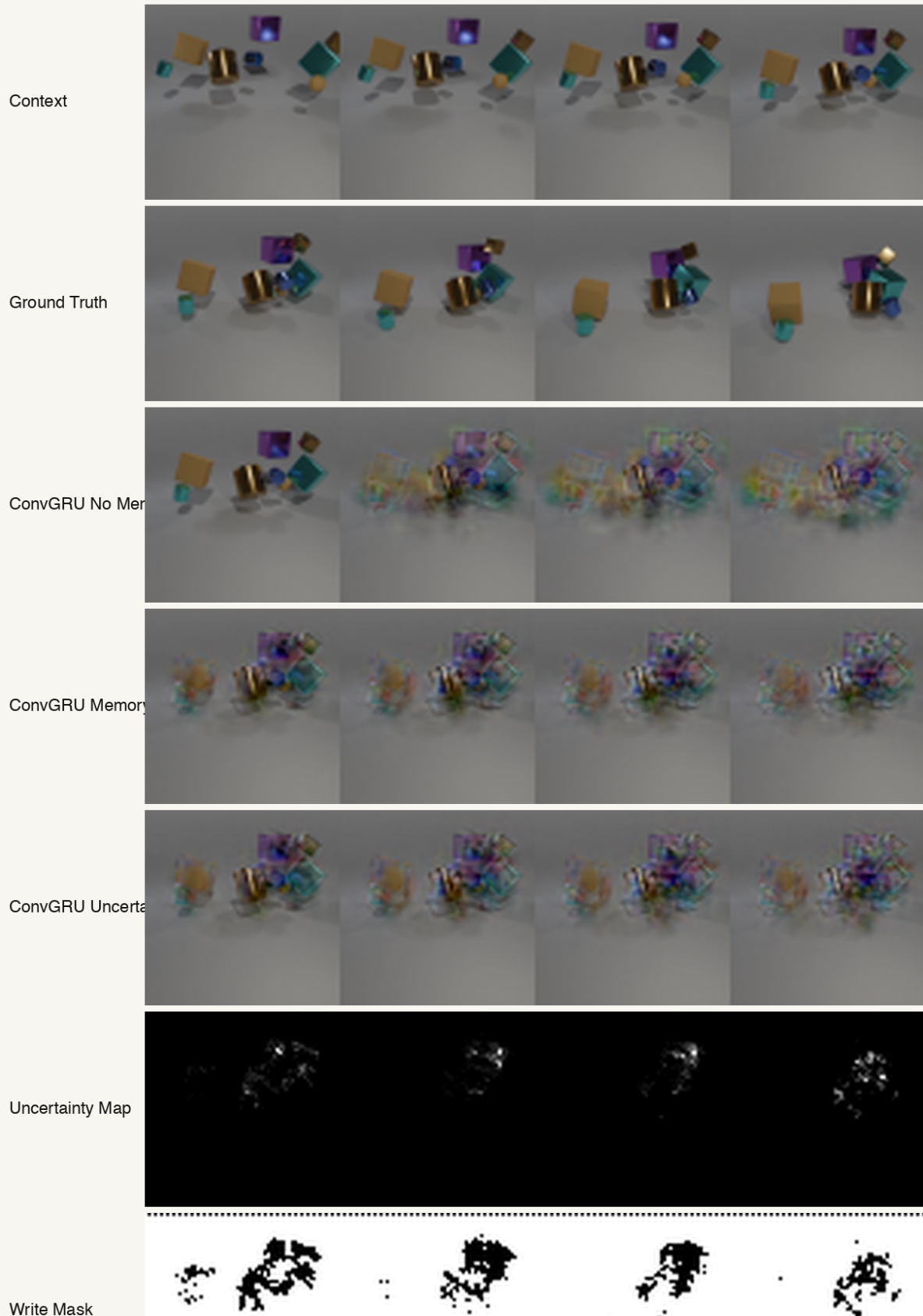


Figure 2: Representative (median) validation window. Although predictions remain blurry at

high_motion: 00006.npz @ frame 1
Representative high-motion example.

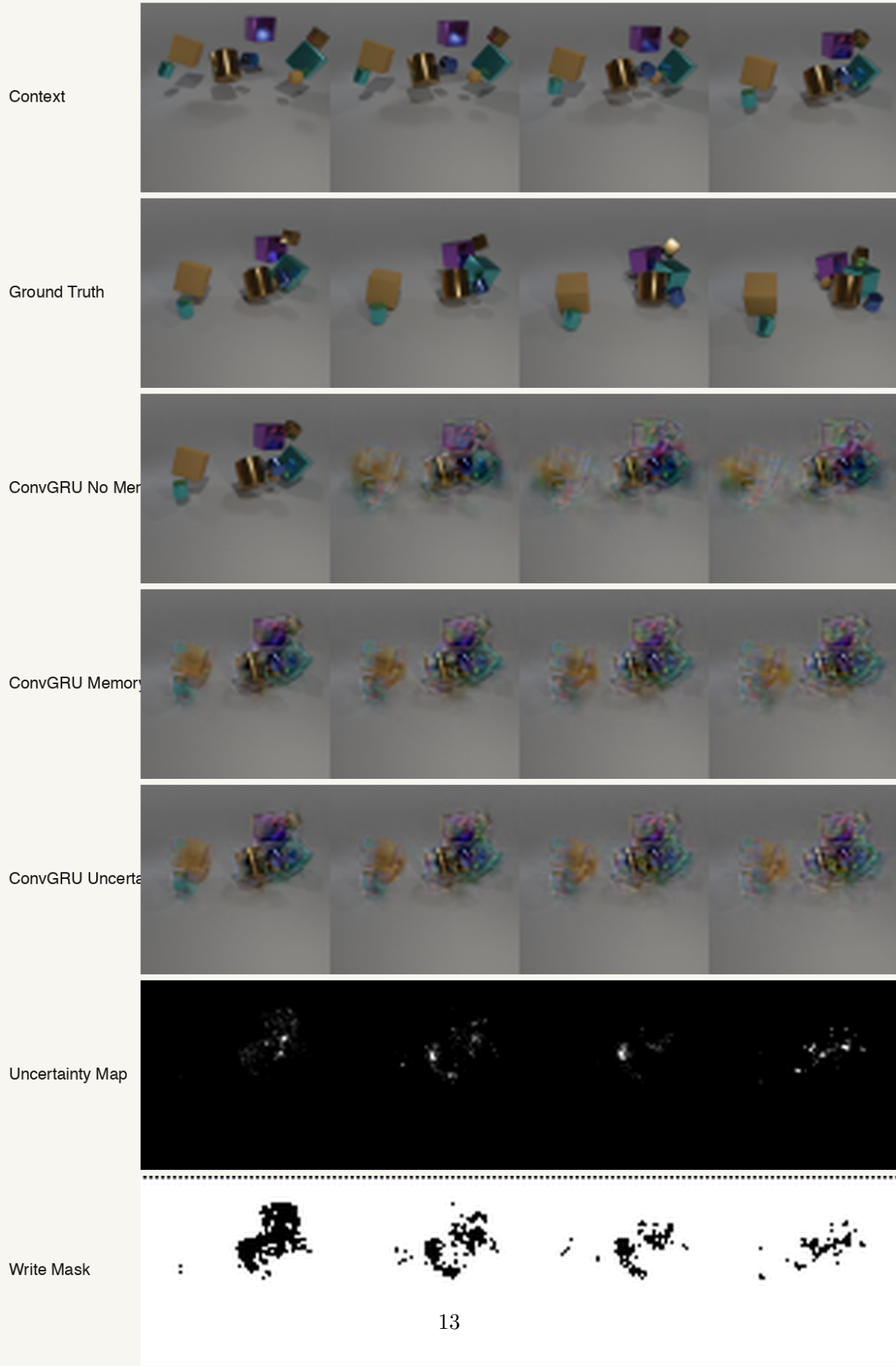


Figure 3: High-motion slice. Same caveats as Figure 2; trends mirror lower dynamic L1 in Table 3.

occlusion_recovery: 00017.npz @ frame 1
Representative occlusion/reappearance example.

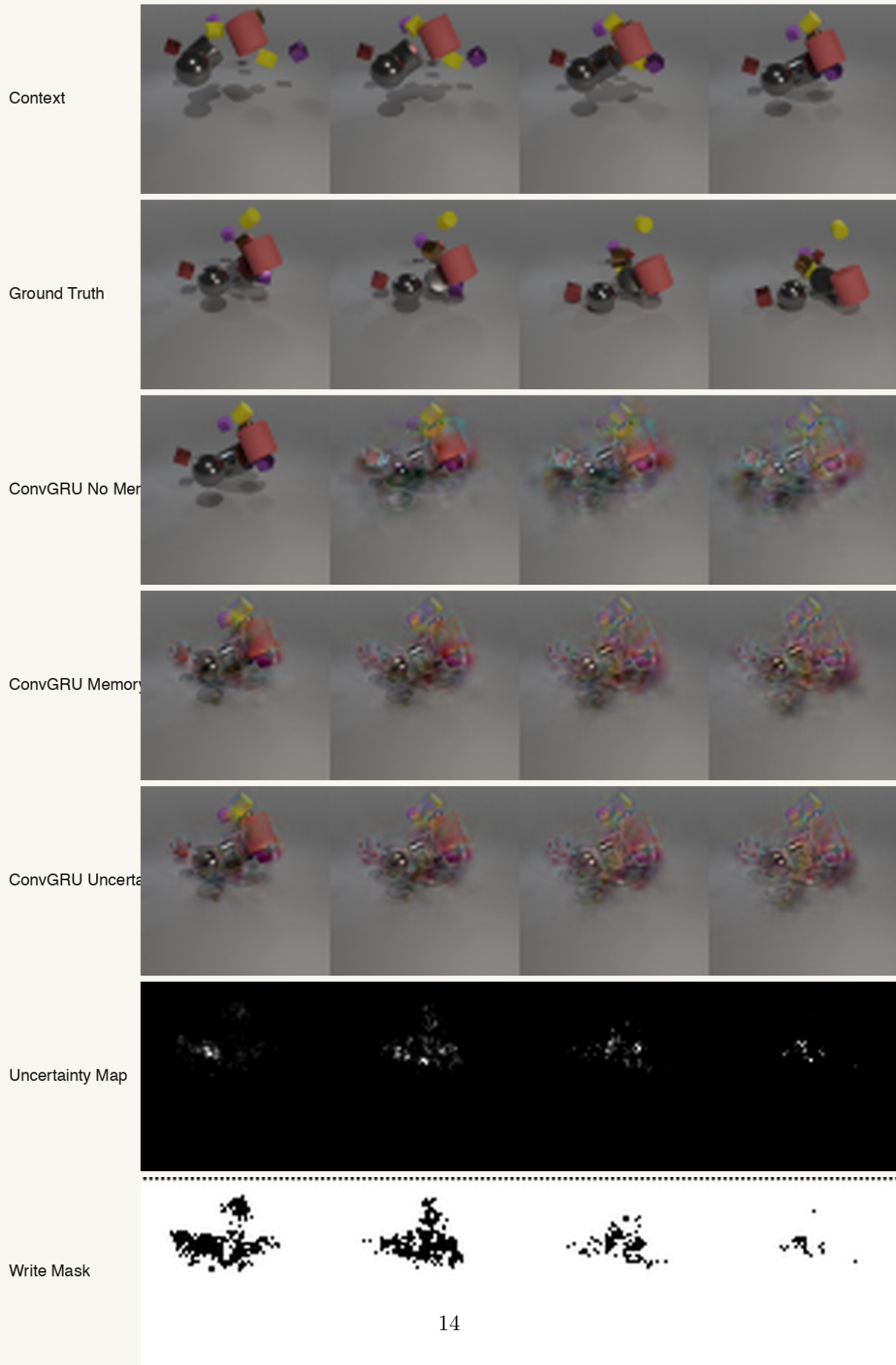


Figure 4: Occlusion-recovery slice. Blur remains visible; nonetheless ordering across models follows

high_memory_coverage: 00006.npz @ frame 2
Representative high-memory-coverage example.

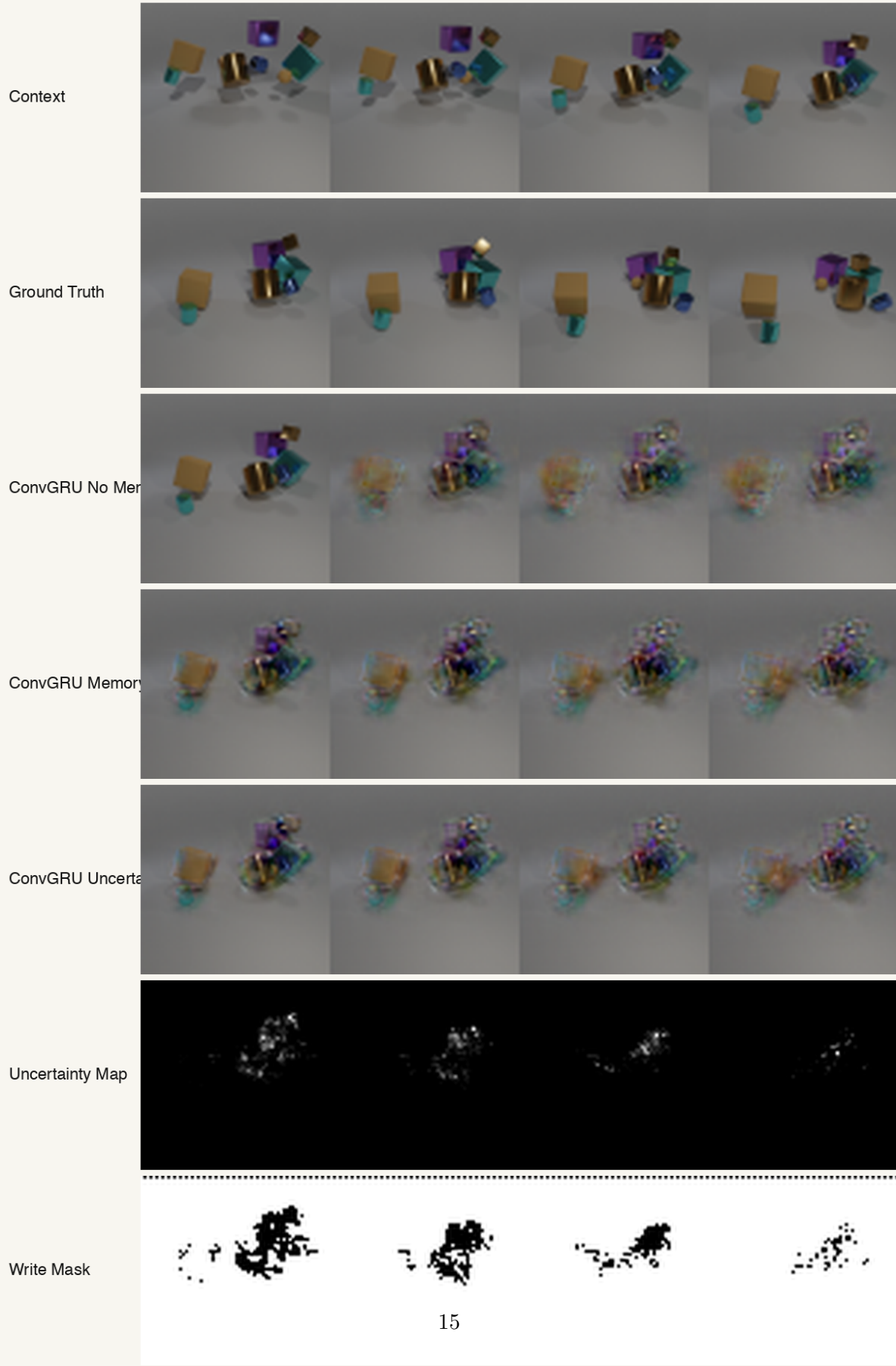


Figure 5: High memory-coverage slice, where memory renders carry the most signal. Uncertainty

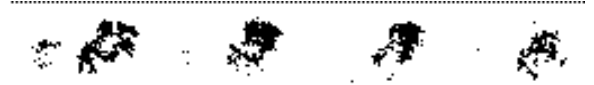
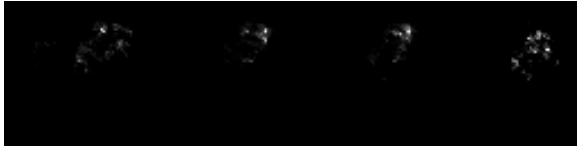


Figure 6: Left: predicted confidence (higher intensity = higher confidence). Right: write gate (white = allow generated-frame memory update, black = hold previous voxel). Together with Table 3, this illustrates how gating concentrates around ambiguous regions while keeping overall write coverage high.